TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Modélisation UML

Chapitre 3: Le Diagramme de Classes

Auteur :

Mme Ines **AGREBI**

UNIFIED MODELING LANGUAGE™

Introduction

- Le diagramme de classe exprime la structure statique du système en termes de classes et de relations entre ces classes.

- L'intérêt des diagrammes de classes est de modéliser les entités du système d'information. Il permet de donner la représentation statique du système à développer.

- Le diagramme met en évidence d'éventuelles relations entre ces classes.

Introduction

- La classe décrit le domaine de définition d'un ensemble d'objets.
- Chaque objet appartient à une classe.
- Les généralités sont contenues dans la classe et les particularités dans les objets.
- Chaque classe est décrite par les données et par les traitements dont elle est responsable pour elle-même et vis-à-vis des autres classes.
- Les traitements sont matérialisés par des opérations (méthodes).

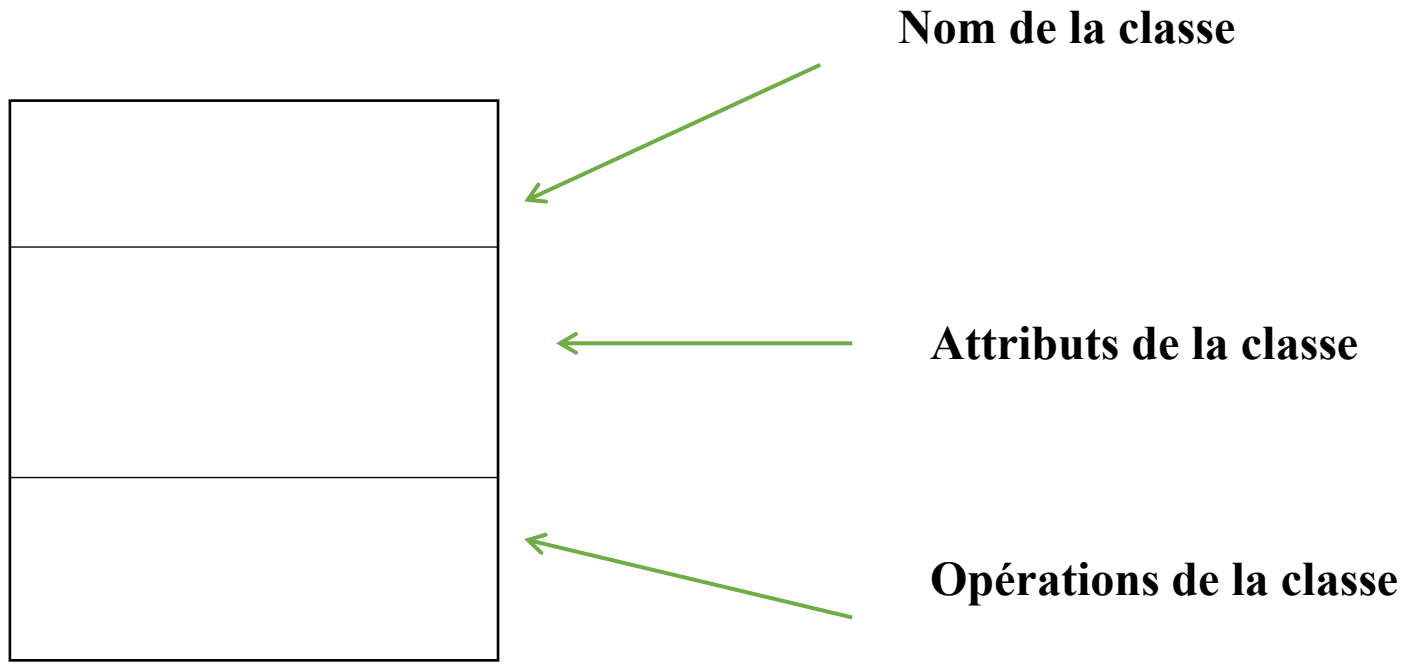
Concepts de base du diagrammes de classes

1- Classe:

- Une classe est une description abstraite d'un ensemble d'objets du domaine d'application; elle définit:
  - Leur structure
  - Leur comportement
  - Leurs relations

- Une classe est un ensemble d'objets ayant les mêmes propriétés ou attributs, un même comportement (opération) et une sémantique commune.

Formalisme:


Nom de la classe
Attributs de la classe
Opérations de la classe

Concepts de base du diagrammes de classes

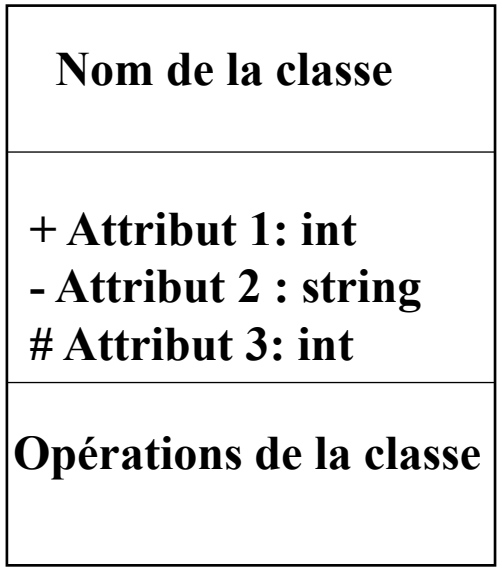
2- Notion d'Attribut:

- Une classe correspond à un concept global d'informations et se compose d'un ensemble d'informations élémentaires, appelés ***attributs de classe***.

- Un attribut représente la modélisation d'une information élémentaire représentée par son nom et son format.

- Un attribut est une propriété élémentaire d'une classe, pour chaque objet d'une classe l'attribut prend une valeur.

Formalisme:


Nom de la classe
+ Attribut 1: int
- Attribut 2 : string
Attribut 3: int
Opérations de la classe

❖ ***Niveaux de visibilité d'un attribut:***

❑ Il existe trois niveaux de visibilités pour les attributs:

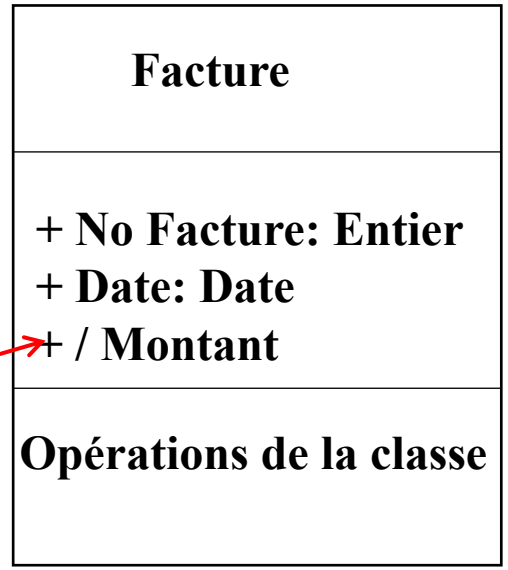
- ***Public (+):*** L'élément est visible pour tous les clients de la classe.
- ***Protégé (#):*** L'élément est visible pour les sous classes de la classe.
- ***Privé (-):*** L'élément n'est visible que par les objets de la classe dans laquelle il est déclaré

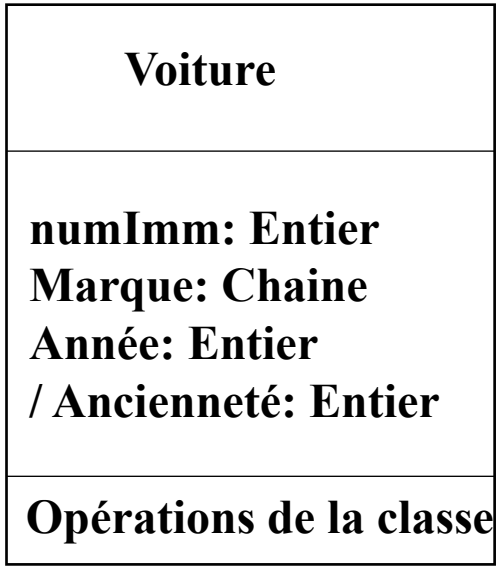
Concepts de base du diagrammes de classes

3- La notion d'identifiant:

- L'identifiant est un attribut particulier, qui permet de repérer de façon unique chaque objet, instance de classe.

On choisit parfois de conserver dans un attribut le résultat d'un calcul effectué à partir d'autre classe ou bien à partir d'autres attributs de la classe.

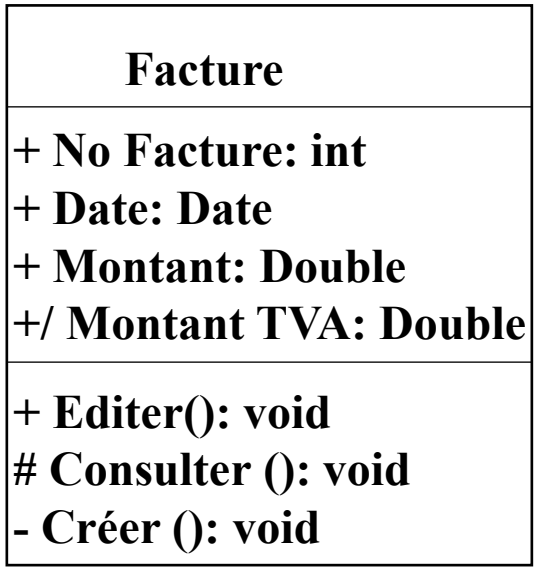




Concepts de base du diagrammes de classes

4- La notion d'opération:

- L'opération représente un élément de comportement des objets, définit de manière globale dans la classe.
- Une opération est une fonctionnalité assurée par une classe qui est appliquée aux objets d'une classe.



❖ ***Niveaux de visibilité d'une opération:***

❑ Il existe trois niveaux de visibilités pour les opérations:

- ***Public (+):*** L'opération est visible pour tous les clients de la classe.
- ***Protégé (#):*** L'opération est visible pour les sous classes de la classe.
- ***Privé (-):*** L'opération n'est visible que par les objets de la classe dans laquelle il est déclaré

Concepts de base du diagrammes de classes

5- La notion de relation:

- S'il existe des liens entre objets, cela se traduit nécessairement par des relations qui existent entre leurs classes respectives.

- Les liens entre les objets doivent être considérés comme des instances de relations entre classes.

- Il existe plusieurs types de relations entre classes:
  - Association
  - Généralisation/spécialisation…

5.1- Lien et Association

- Un lien permet d'établir une relation entre objets.

- Une association est une relation qui relie plusieurs classes entre elles.

- Elle existe entre les classes; elle est introduite pour montrer une structure et non des échanges de données.

- Chaque classe qui participe à l'association joue un rôle qui est définit par la notion de ***Multiplicité***.

Multiplicité ou Cardinalité

- La multiplicité définit le nombre d'instances de l'association pour une instance de la classe.
- Les valeurs de multiplicité expriment les contraintes liées au domaine de l'application.
- Il est important de déterminer les valeurs de multiplicité optimales pour trouver le bon équilibre entre complexité et efficacité.

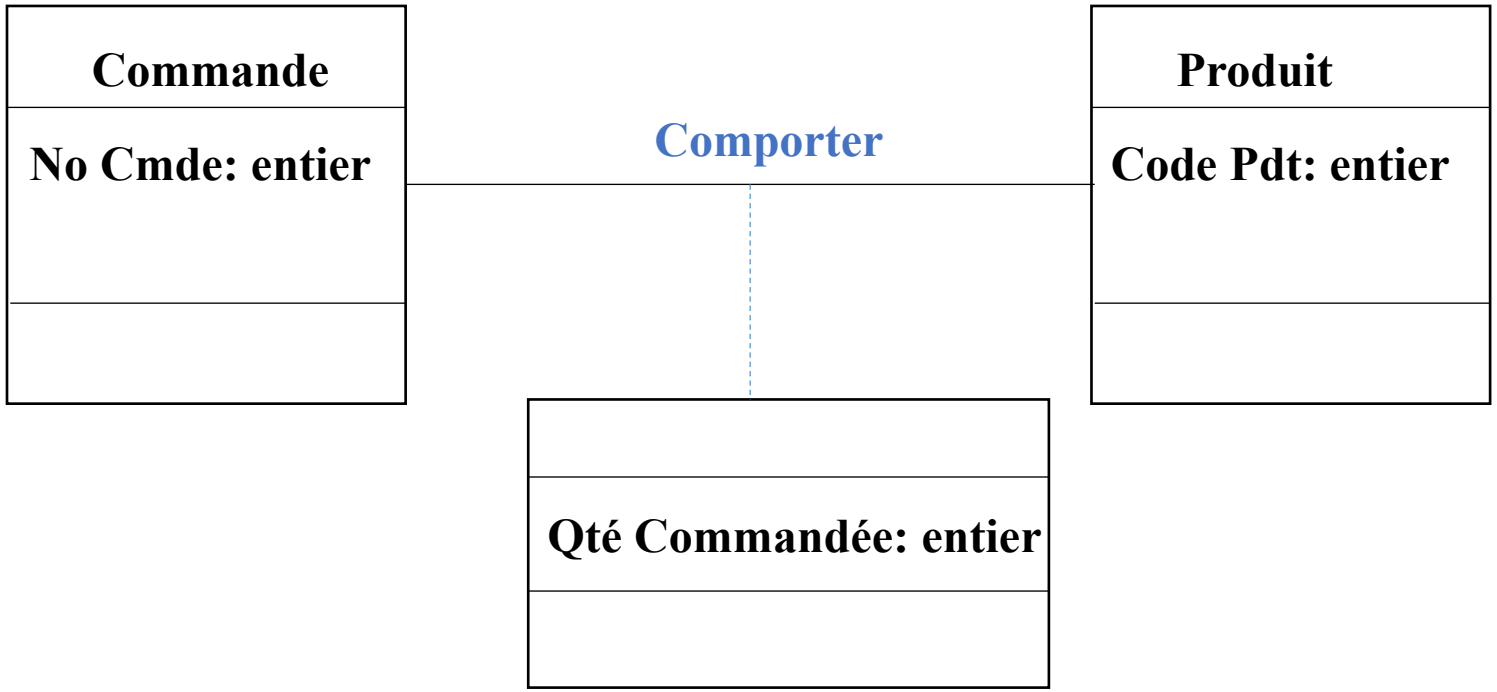
- **1:** Un et un seul
- **0..1:** Zéro ou 1
- **N** ou ***:** N (entier naturel)
- **M..N:** De M à N entiers naturels
- **0..*:** De 0 à plusieurs
- **1..*:** De 1 à plusieurs

Formalisme et exemple: (Nom de l'association et multiplicité des rôles)



5.2- Classe-association:

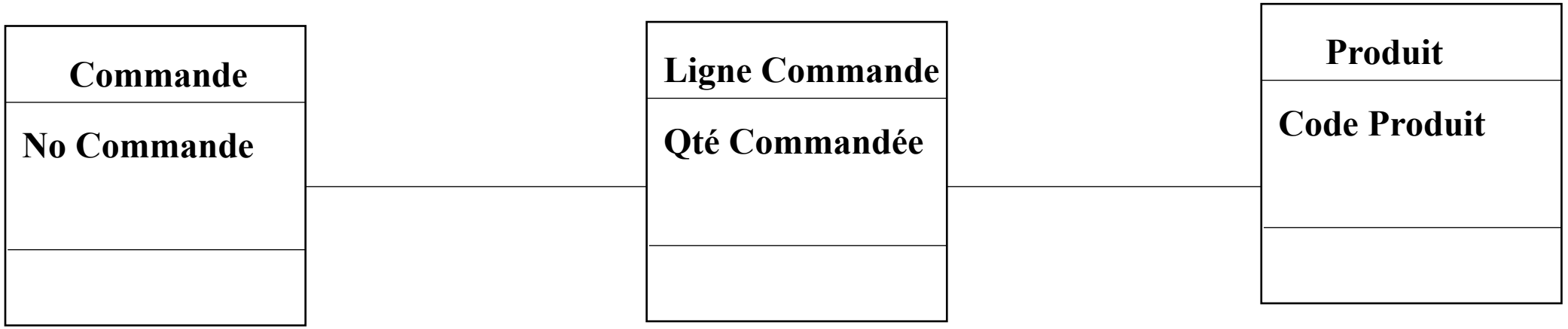
- Une association porteuse d'attribut est dite classe-association.
- ***Exemple:***



➔L'attribut « Qté Commandée » dépend fonctionnellement de numéro de la commande et du code produit, on va donc placer l'attribut « Quantité Commandée » dans l'association Comporter.

Remarque 1:

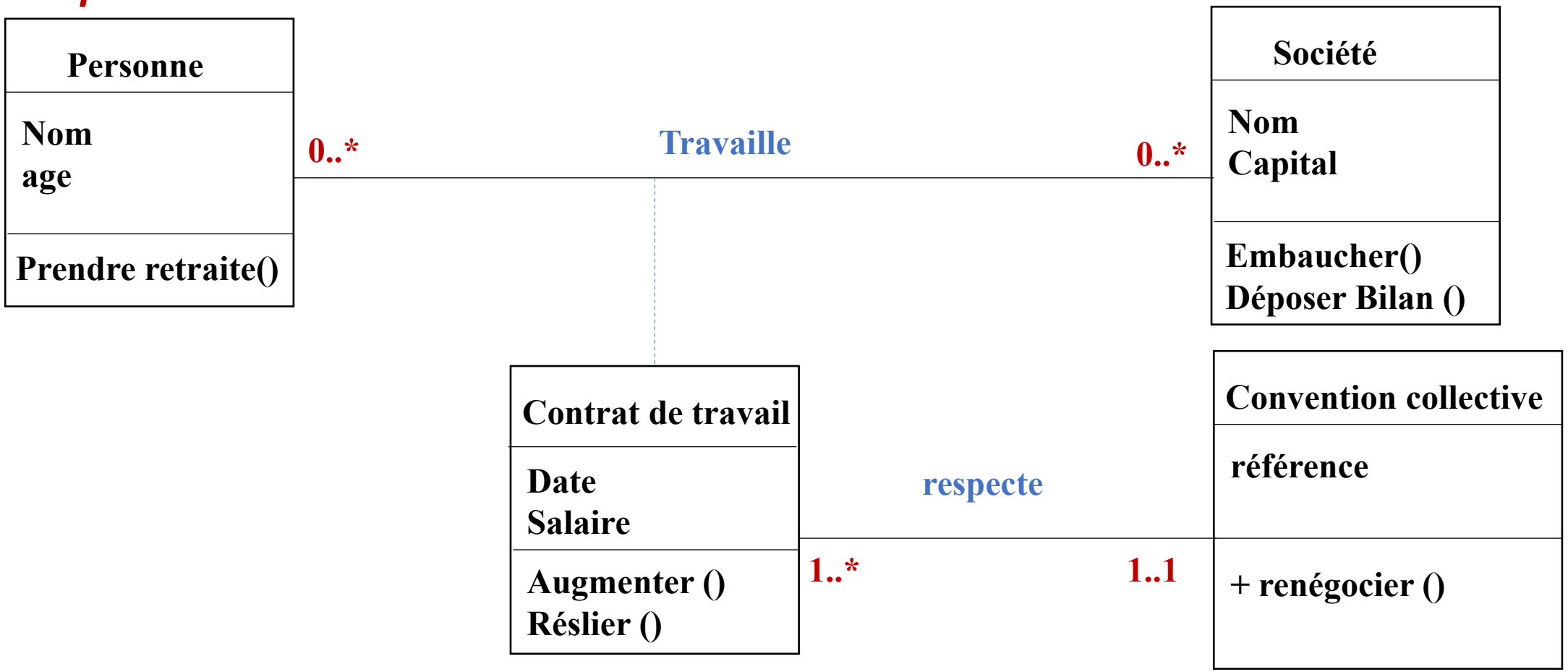
- Toute classe-association peut être remplacée par une classe intermédiaire et qui sert de pivot pour une paire d'association.



Remarque 2:

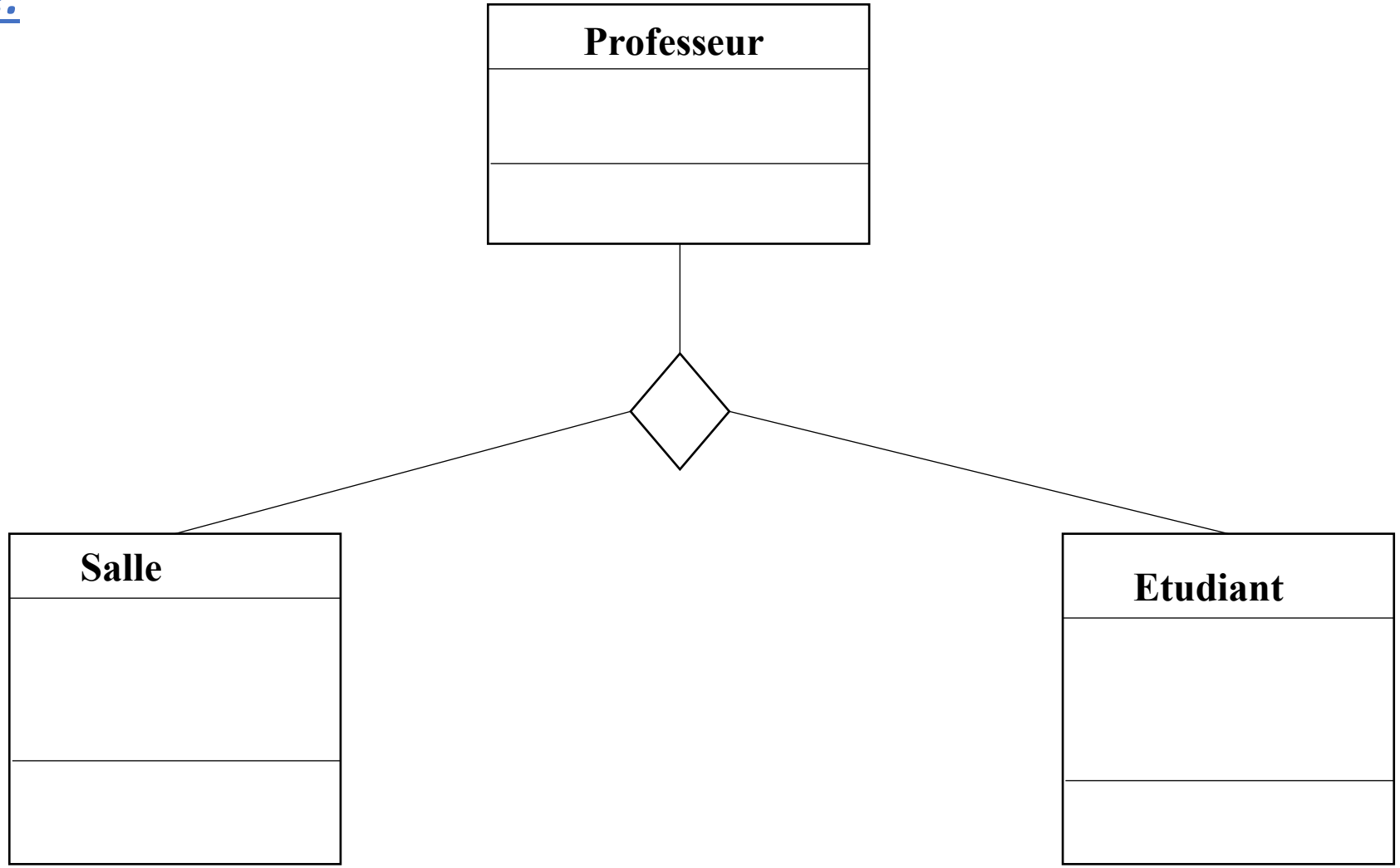
- Une classe d'association est un élément qui peut avoir à la fois les propriétés d'une classe et d'une association.

Exemple:

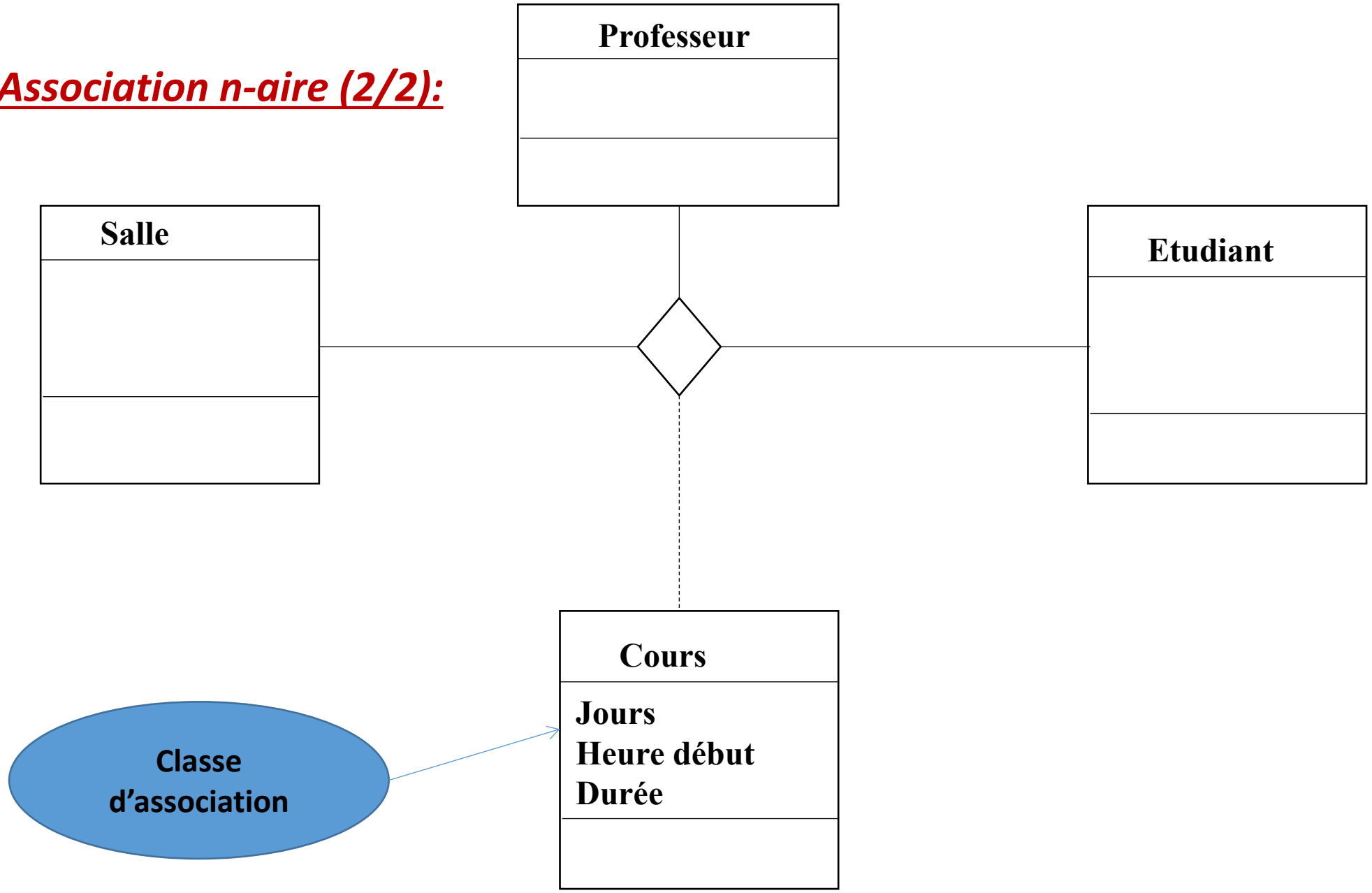


5.3- Association n-aire (1/2):

- C’est une association entre 3 classes ou plus.
- ***Exemple:***



5.3- Association n-aire (2/2):



5.4- Agrégation (1/2):

- C’est une association qui porte les sémantiques suivantes:
  - Composé et composants
  - Tout et parties
  - Contenu et contenant
- Les agrégations n’ont pas besoin d’être nommées implicitement.
- Elle signifie « contient » ou « composé de ».



- La classe C1 est Composé de la classe C2.
- Le losange doit être représenté du coté de l’agrégation.

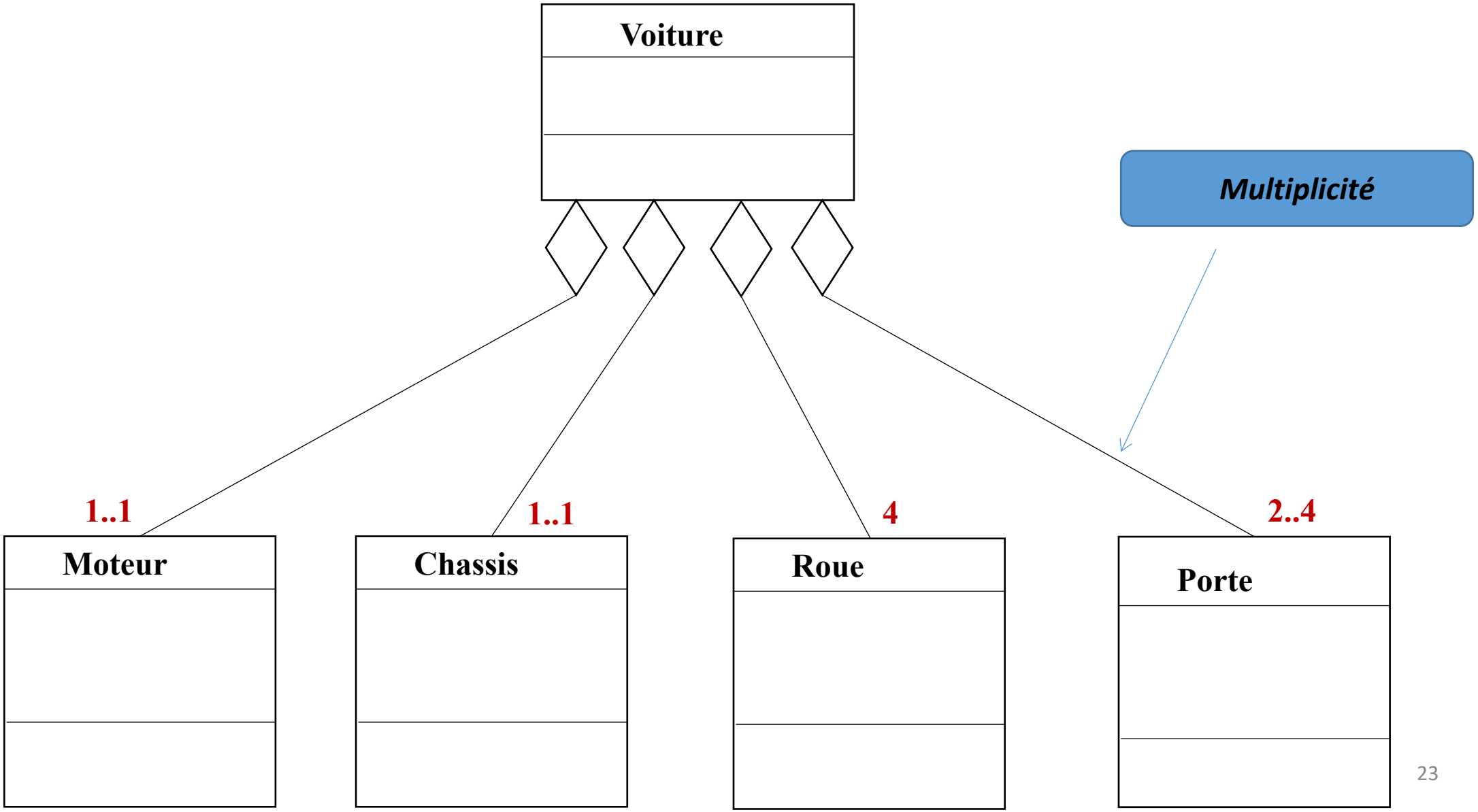
- La notion de composition ou agrégation: Il s'agit d'une relation entre deux classes, spécifiant que les objets d'une classe sont des composants de l'autre classe.

- Une relation d'agrégation permet donc de définir des objets composés d'autres objets.

- L'agrégation permet d'assembler des objets de base, afin de construire des objets plus complexes.

- La composition consiste à réunir des objets pour en former un autre.

❑ *Exemple d'Agrégation:*



5.4- Compositions (2/2):

- ***Composition (Forte):***

- Est une relation d'agrégation dont laquelle il existe une contrainte de durée de vie entre la classe composant et la ou les classes composées.
- La destruction de l'objet composé entraine la destruction de ses composants

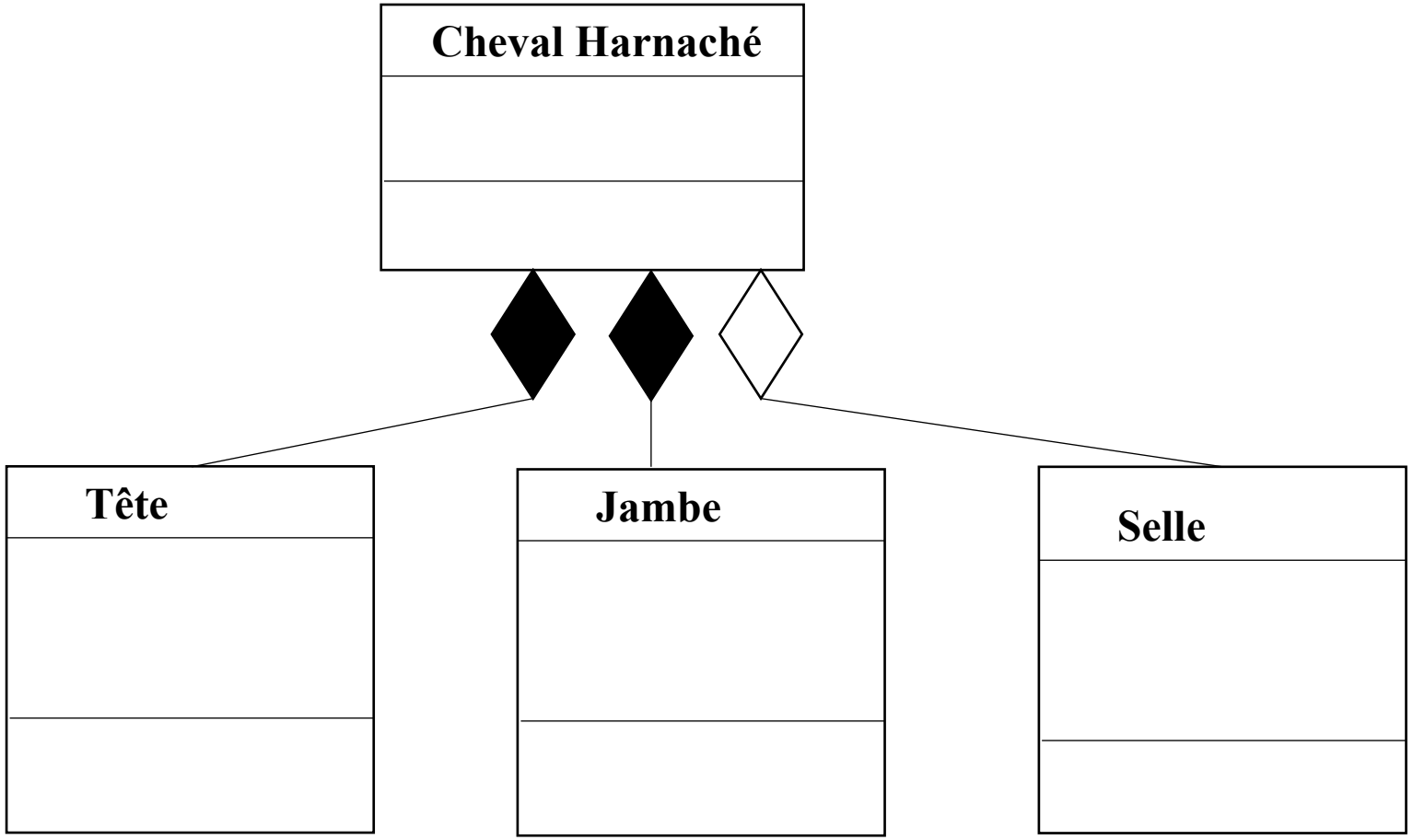


- ***<u>Composition Faible (Agrégation):</u>*** Les composants peuvent être partagés entre plusieurs objets complexes.



Exemple d'agrégation:

- Un cheval est un exemple d'objet complexe. Il est constitué de ses différents organes (tête, jambes…).
- La représentation graphique de cette composition est la suivante:



Explication de l'exemple:

- Composition Faible: Les composants peuvent être partagés entre plusieurs objets complexes (dans notre exemple l'agrégation ou composition faible pour la selle).

- Composition Forte: La destruction de l'objet composé entraine la destruction de ses composants (Jambe et tête du cheval dans notre exemple ne peuvent pas être partagés et la disparition du cheval entraîne la disparition de ses organes).

5.5- Contraintes sur les associations (1/5):

❑Il y a des contraintes peuvent être ajoutés sur les associations:

1. La contrainte d'inclusion {sous ensemble}: {subset}

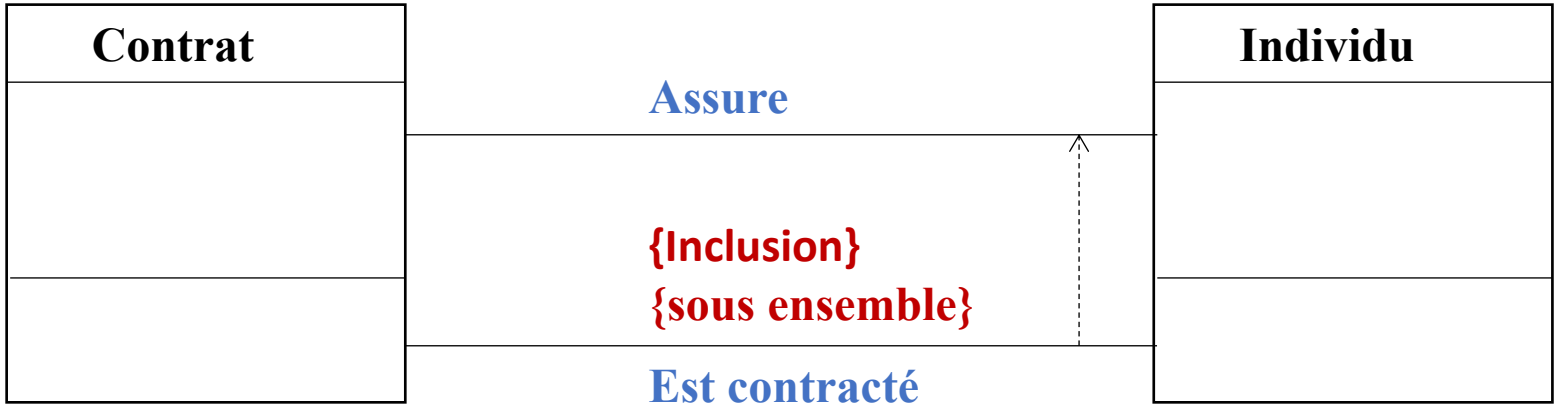
- Elle indique une collection est incluse dans un autre ensemble.
- Nécessite la présence d'au moins deux relations
- Dans cette exemple, les personnes qui jouent le rôle de délégué font partie des personnes qui jouent le rôle d'élèves.



Exemple:

1. La contrainte d'inclusion {sous ensemble}: {subset}

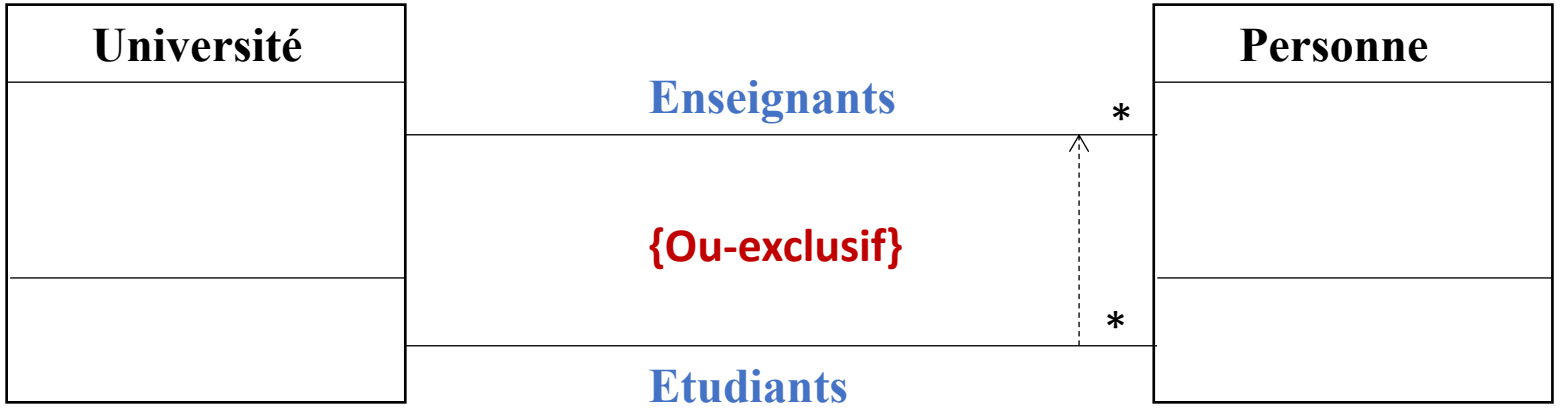
- Elle permet de préciser qu'une classe est incluse dans une autre, la flèche de dépendance indique le sens de la contrainte.
- Dans cette exemple, le contractant d'un contrat fait obligatoirement partie des individus assurés.

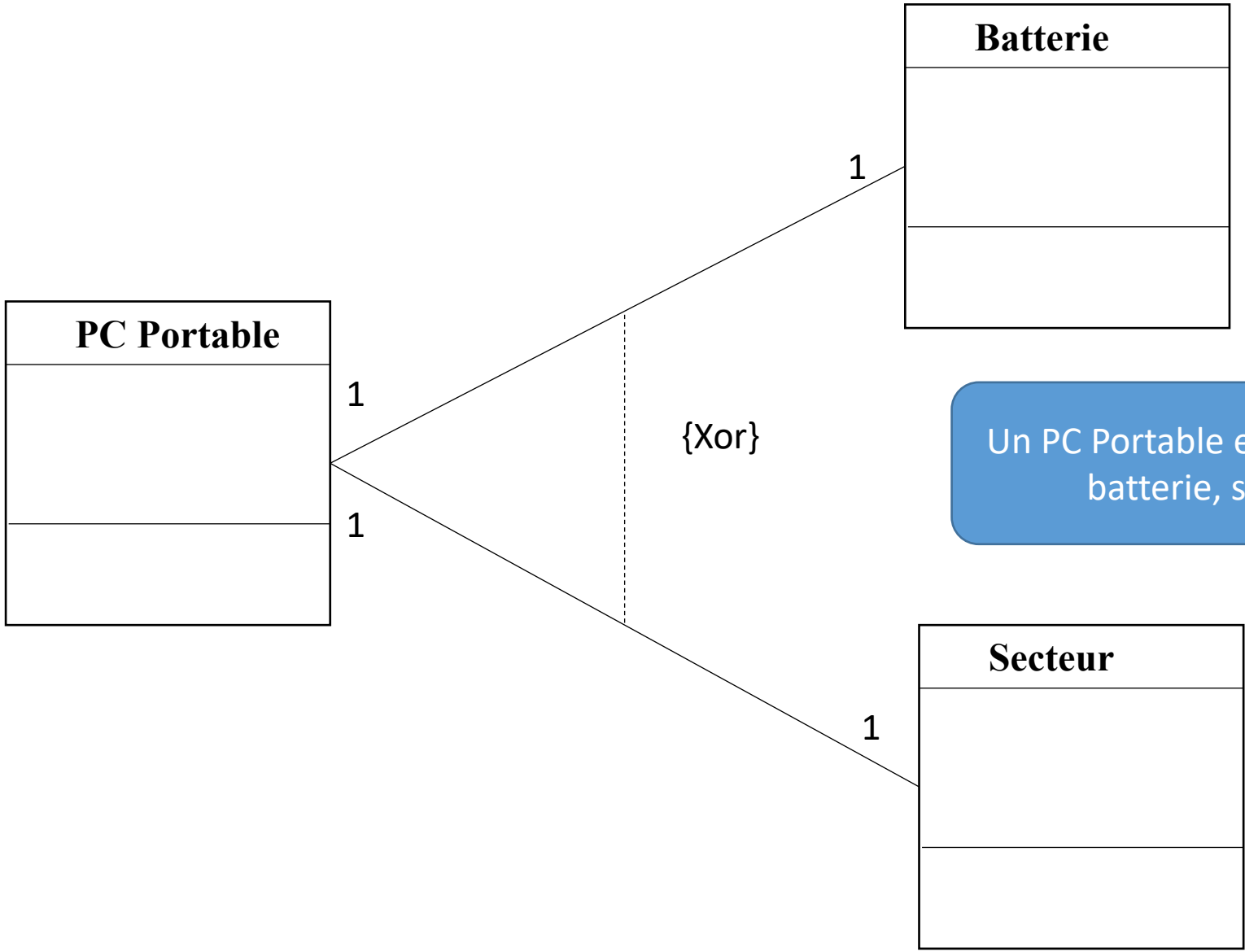


5.5- Contraintes sur les associations (2/5):

2. La contrainte {ou-exclusif}: {Xor}

- Cette contrainte précise que pour un objet donné, une seule association parmi un groupe d'association est valide
- Indique que parmi un groupe d'associations, une seule est valide à la fois →pas de simultanéité.



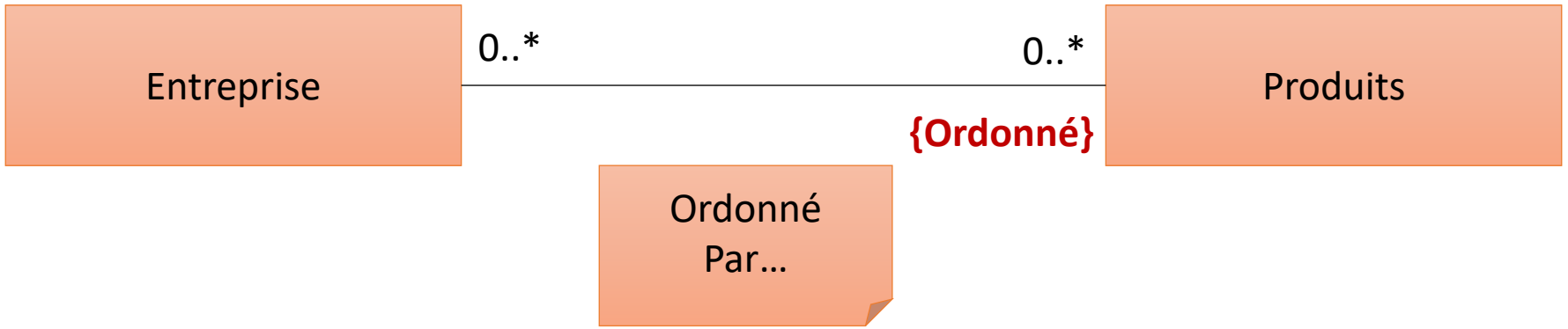


Un PC Portable est alimenté soit à partir d'une batterie, soit à partir d'un secteur

5.5- Contraintes sur les associations (3/5):

3. La contrainte {ordonné}:

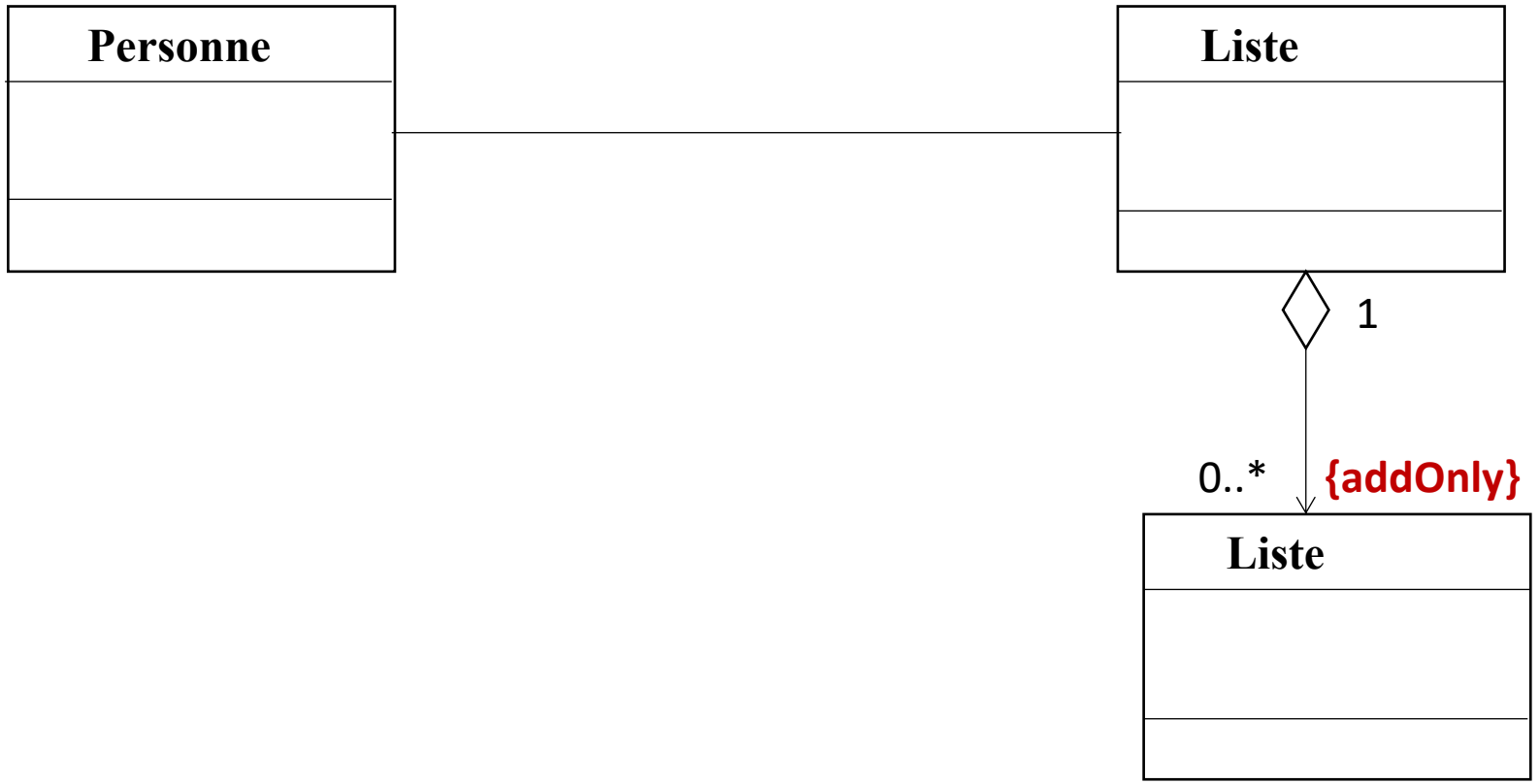
- Indique que les objets seront ordonnés à chaque opération de : création, modification, suppression…
- Le modèle ne spécifie pas comment les objets sont ordonnés => On utilise un commentaire.



5.5- Contraintes sur les associations (4/5):

4. La contrainte {addOnly}:

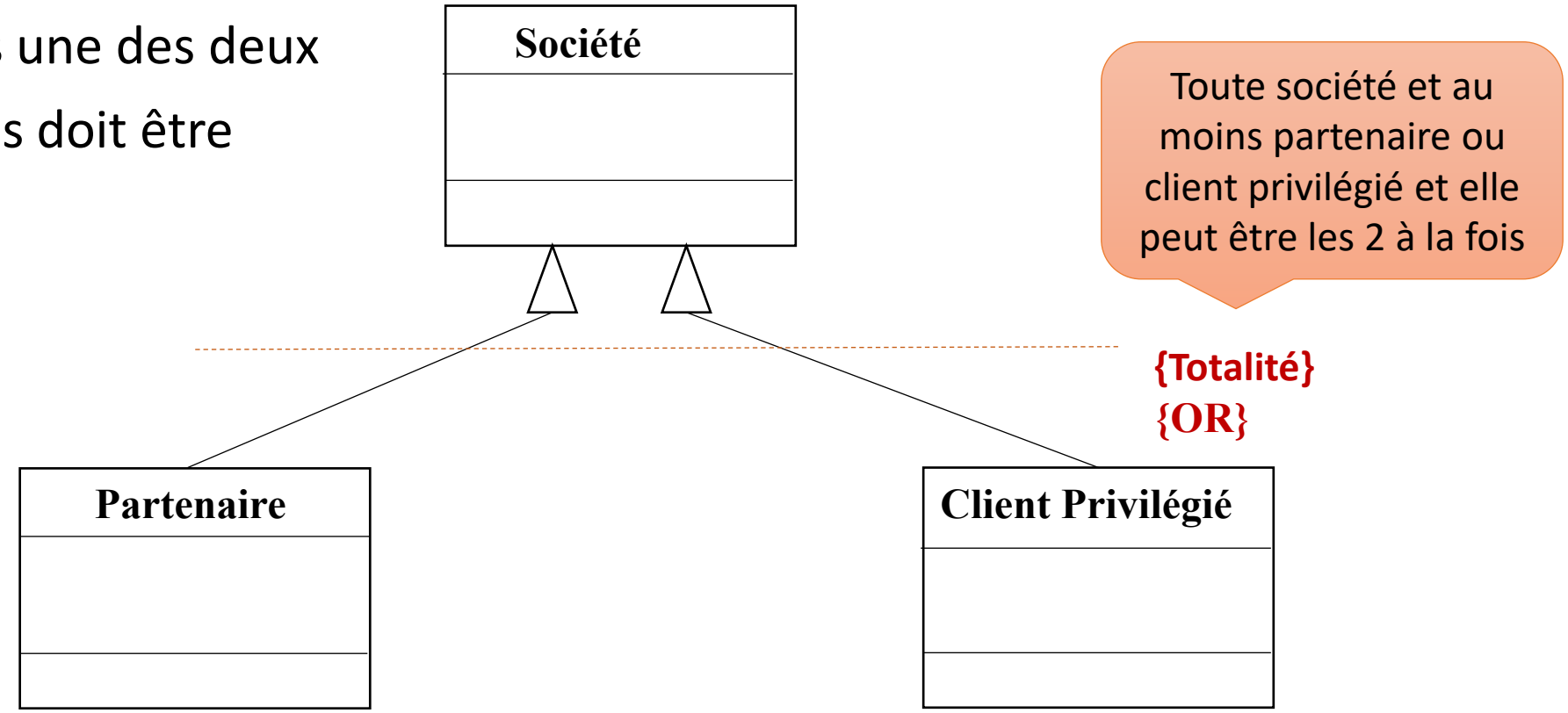
- La contrainte prédéfinie {addOnly} autorise l'ajout de nouveaux objets, mais pas leur suppression ni leur mise à jour.



5.5- Contraintes sur les associations (5/5):

5. La contrainte de Totalité {OR}:

- La contrainte prédéfinie {OR} signifie que toutes les instances d'une classe correspondent au moins à une des instances des classes liées.
- Au moins une des deux associations doit être instanciée.



5.6- Concept de généralisation/spécialisation (1/2):

- Le principe de généralisation/spécialisation permet d'identifier parmi les objets d'une classe (générique) des sous-ensembles d'objets (des classes spécialisées) ayant des définitions spécifiques.

- La classe plus spécifique (fille, dérivée, spécialisée, descendante…) est cohérente avec la classe plus générale (classe mère, générale…), c'est-à-dire qu'elle contient par héritage tous les attributs, membres, relations de la classe générale et peut contenir d'autres.

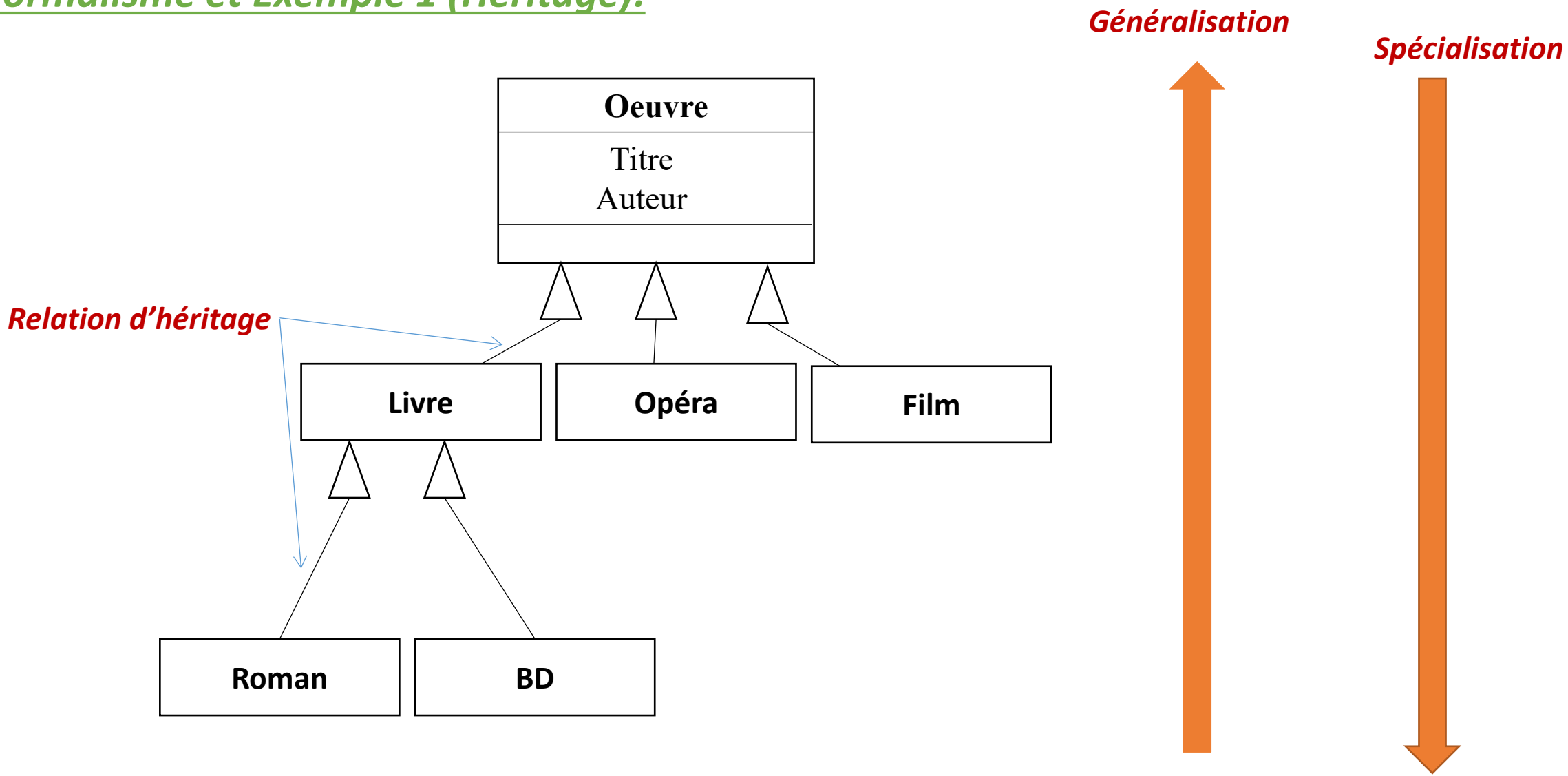
- L'héritage est un mécanisme de transmission des propriétés d'une classe (ses attributs et méthodes) vers une sous-classe. Il est destiné à exprimer les similitudes entre les classes.

5.6- Concept de généralisation/spécialisation (2/2):

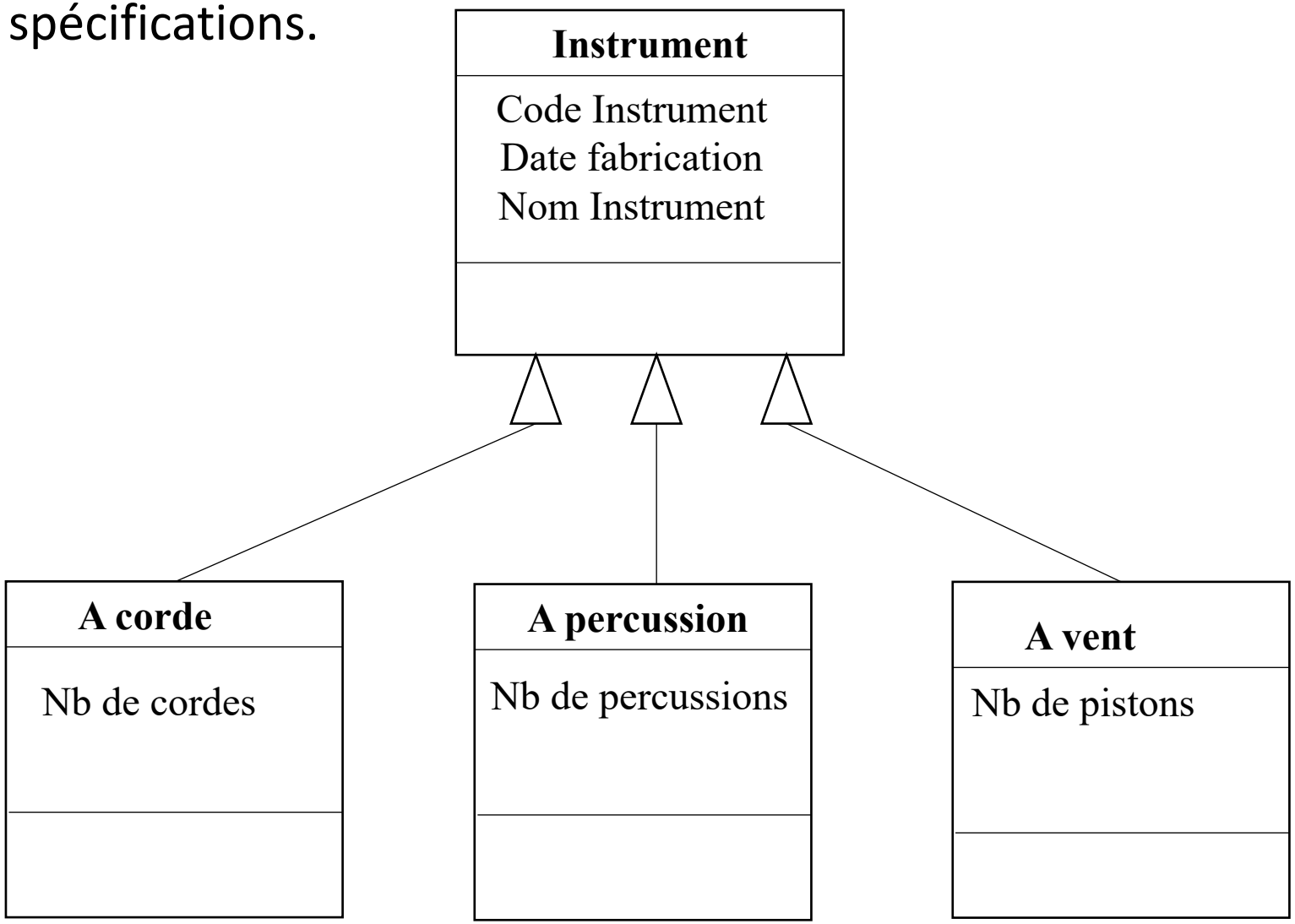
- La classe fille hérite des caractéristiques de sa classe mère (attributs et méthodes) mais elle se distingue par ses propres caractéristiques.

- Une classe peut être spécialisée en d'autres classes, afin d'y ajouter des caractéristiques spécifiques ou d'en adopter certaines.

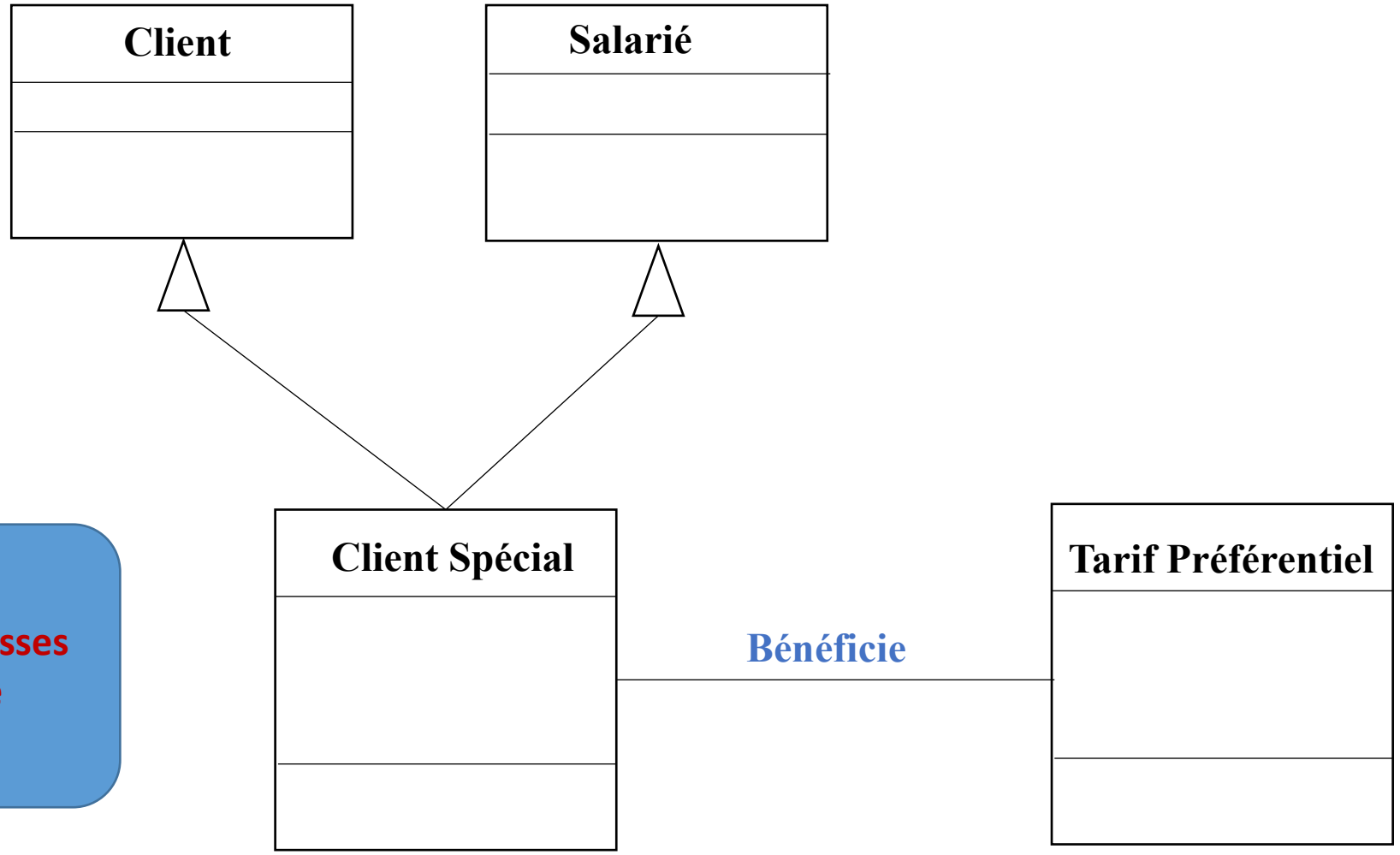
Formalisme et Exemple 1 (Héritage):



Formalisme et Exemple 2 (Héritage): Une classe fille n'a qu'une seule classe mère et elle hérite d'une partie des attributs et méthodes de celle-ci en ayant ses propres spécifications.



Exemple 2 (Héritage Multiple): Une classe fille hérite de plusieurs classes mères. Elle hérite d'une partie des attributs et méthodes de chacune de ses classes mères, en plus de ses propres spécifications.



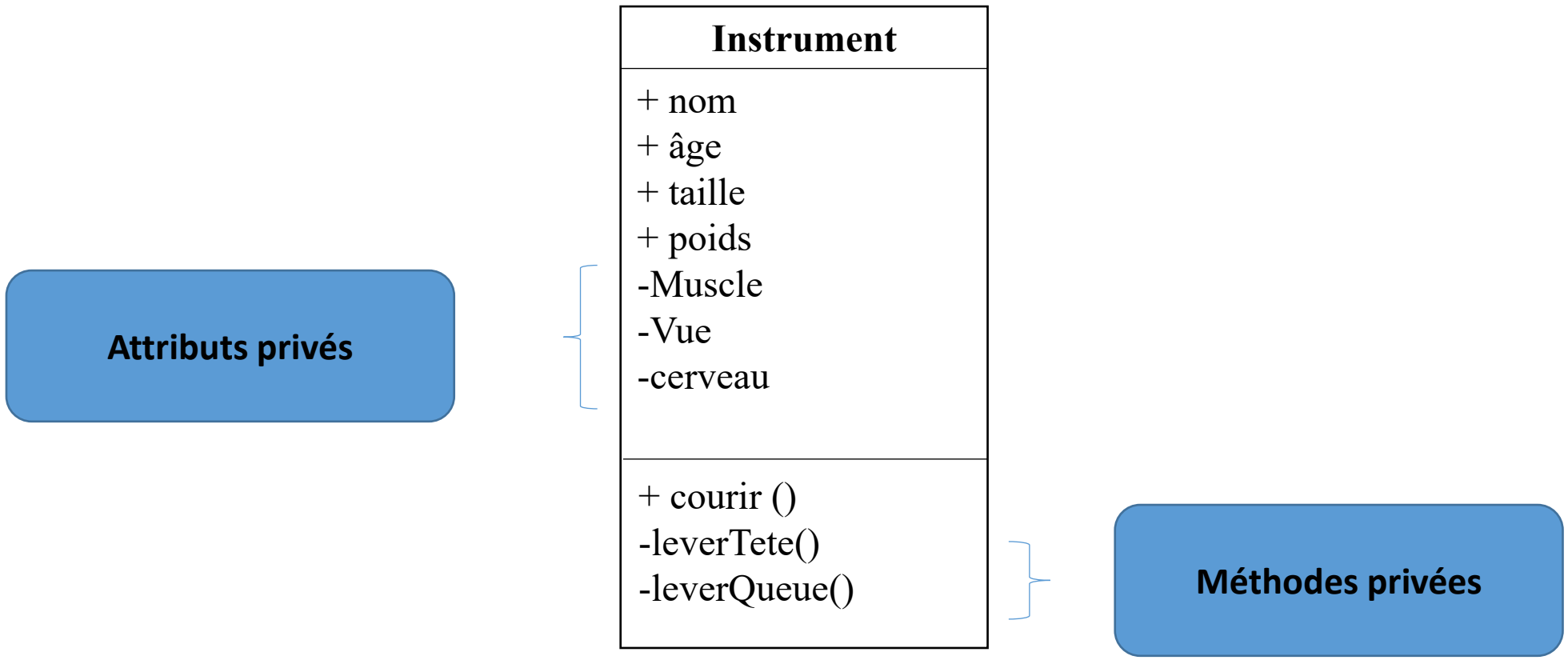
Fusionner plusieurs classes en une seule classe

5.7- Concept d'Encapsulation:

- Le concept d'encapsulation à masquer les détails d'implémentation d'un objet, en définissant une interface.
- L'interface est la vue externe d'un objet, elle définit les services accessibles (offerts) aux utilisateurs de l'objet.
- L'encapsulation facilite l'évolution d'une application car elle stabilise l'utilisation des objets: on peut modifier l'implémentation des attributs d'un objet sans modifier son interface.
- L'encapsulation garantit l'intégrité des données (fournir une précision et une fiabilité de l'information et empêcher toute modification non autorisée), car elle permet d'interdire l'accès direct aux attributs des objets.

- Sépare les aspects externes d'un objet, accessibles aux autres objets, des détails d'implémentation internes, qui leur sont cachés.
- L'encapsulation consiste à masquer des attributs et des méthodes de l'objet vis-à-vis des autres objets.
- En effet, certains attributs et méthodes ont pour seul objectif des traitements internes à l'objet et ne doivent pas être exposés aux objets extérieurs.
- Encapsulés, ils sont appelés les attributs et méthodes privés de l'objet. L'encapsulation est une abstraction puisque l'on simplifie la représentation de l'objet vis-à-vis des objets extérieurs.
- Cette représentation simplifiée est constituée des attributs et méthodes publiques de l'objet.
- La définition de l'encapsulation se fait au niveau de la classe.
- Les objets extérieurs à un objet sont donc les instances des autres classes.

Exemple Encapsulation:

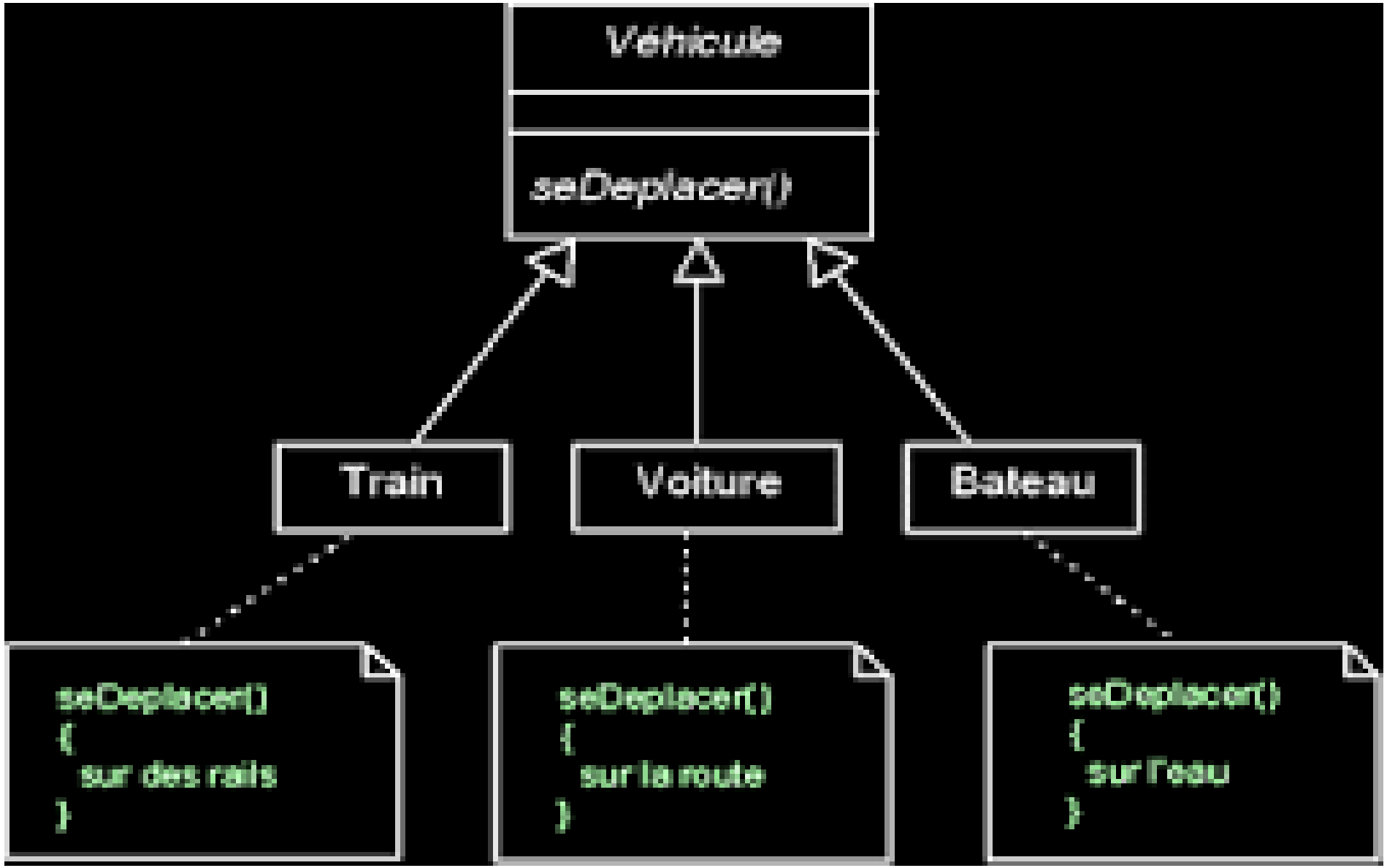


5.8- Concept de polymorphisme:

- Le concept de polymorphisme représente la faculté d'une même opération de s'exécuter différemment suivant le contexte de la classe où elle se trouve.

- Il permet de définir plusieurs formes pour une méthode commune à une hiérarchie d'objets. C'est à l'exécution que l'on détermine quelle forme appeler suivant la classe de l'objet courant.

- Le polymorphisme augmente la généricité du code.


Véhicule
seDeplacer()
Train
Voiture
Bateau
seDeplacer()
{
sur des rails
}
seDeplacer()
{
sur la route
}
seDeplacer()
{
sur l'eau
}

5.9- Concept d'Abstraction:

- L'abstraction est un principe très important en modélisation.

- Elle consiste à retenir uniquement les propriétés pertinentes d'un objet pour un problème précis.

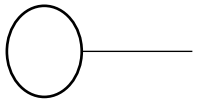
- Les objets utilisés en UML sont des abstractions d'objets du monde réel.

- Permet de s'attacher aux aspects essentiels sans entrer dans les détails, autrement dit de se concentrer sur ce que représente l'objet et sur son comportement avant de décider de la façon de l'implémenter.

Interface

- Une interface définit le comportement visible d'une classe. Ce comportement est défini pour une interface (classe particulière avec le stéréotype « interface »).

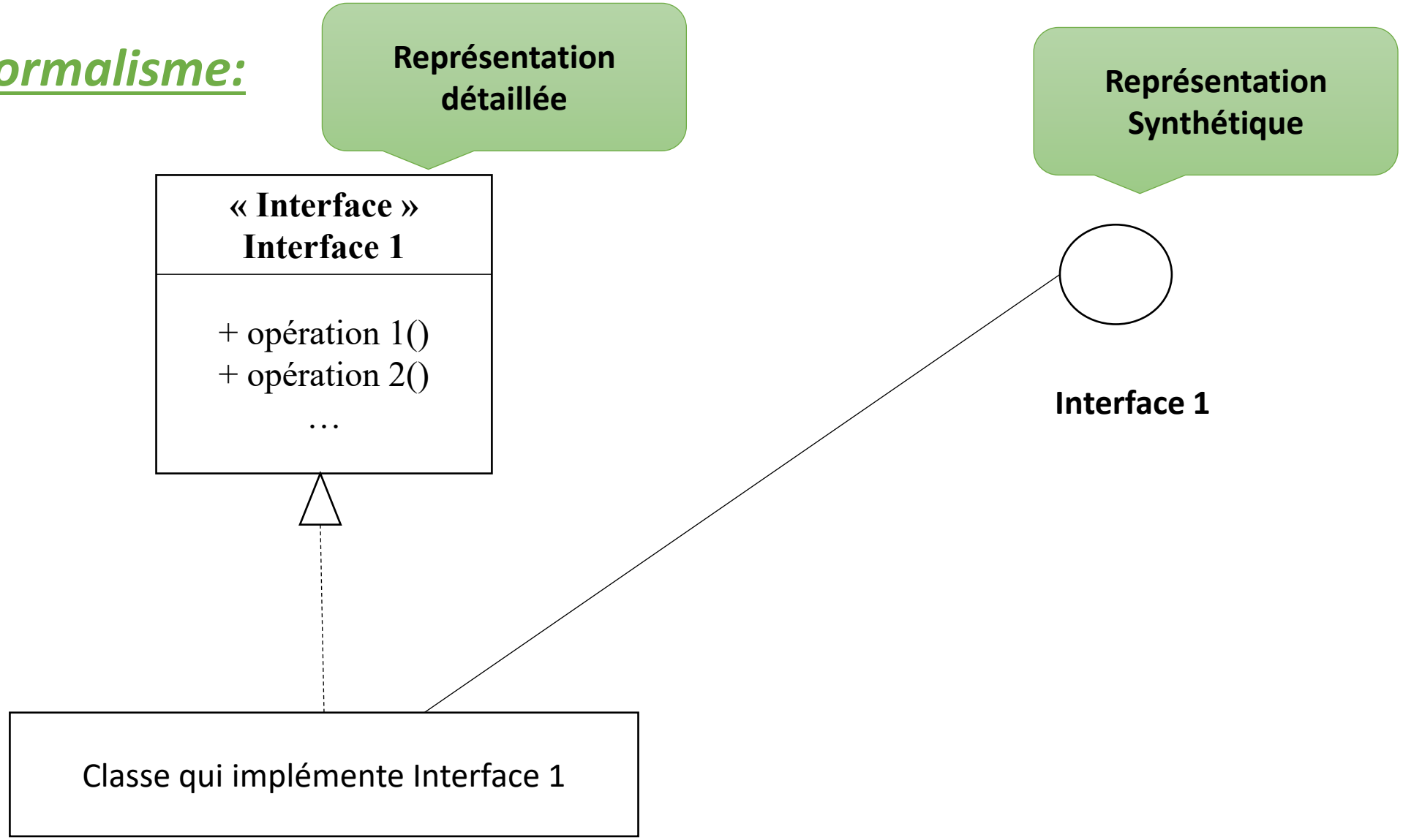
- Dans UML, on représente les interfaces de deux manières soit:

  - Au moyen de petits cercles reliés par un trait à l'élément qui fournit les services décrits par l'interface: 

  - Au moyen de classe avec le mot clé « interface ».

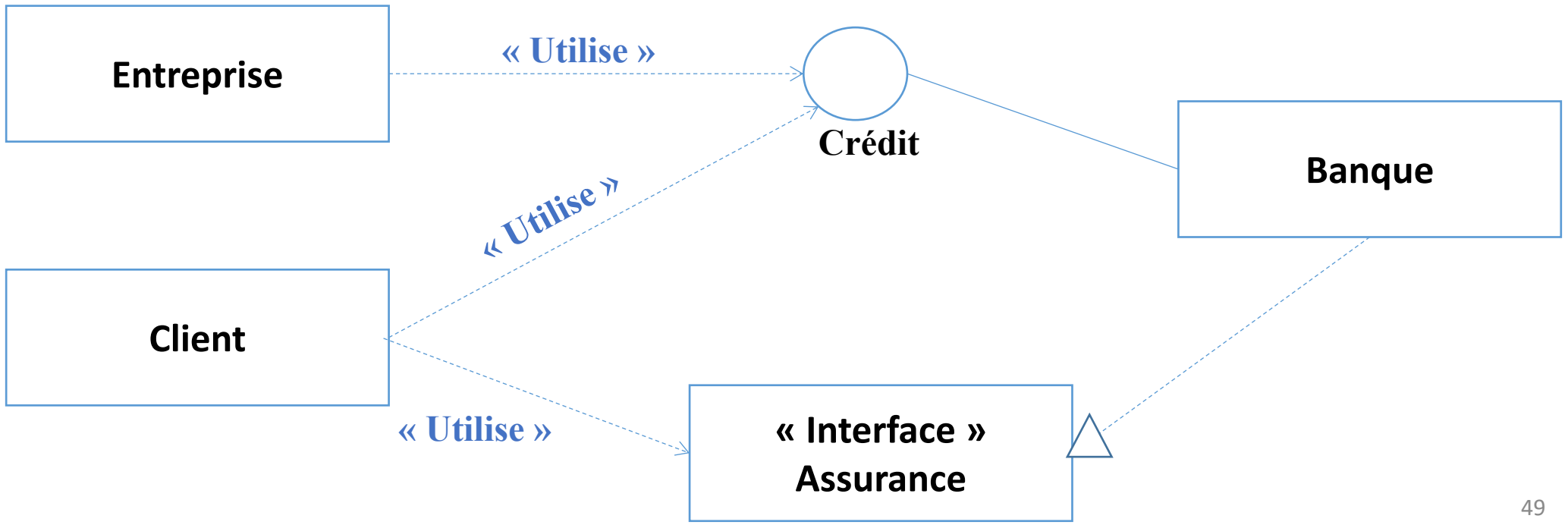
- La classe d’interface est une spécification et non une classe réelle.
- Seules des signatures de méthodes sont présentes dans les interfaces.
- On parle de relation de réalisation entre une interface et une classe qui l’implémente.
- Une classe qui réalise une interface doit définir le corps de toutes ses méthodes abstraites.

Formalisme:


Représentation détaillée
Représentation Synthétique
« Interface »
Interface 1
+ opération 1()
+ opération 2()
…
Interface 1
Classe qui implémente Interface 1

Exemple:

- L'exemple suivant illustre la modélisation de deux interfaces ''Crédit '' et ''Assurance'' d'une classe Banque.
- Une relation indique que la classe Banque réalise l'interface Assurance et Crédit.



Exercice d'application: Etude de cas

Exercice 1:

❑Soit les phrases suivantes:

1- Un répertoire contient des fichiers.

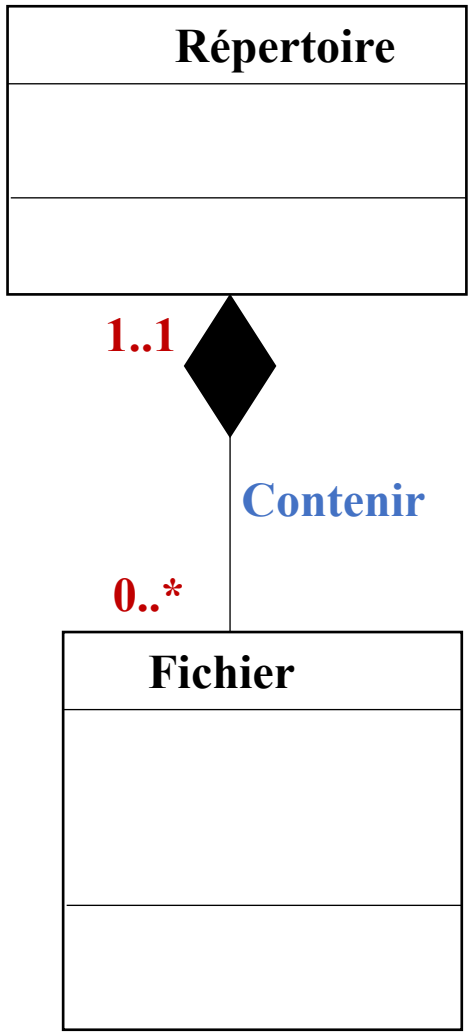
2- Les modems et claviers sont des périphériques d’E/S.

3- Une transaction boursière est achat ou une vente.

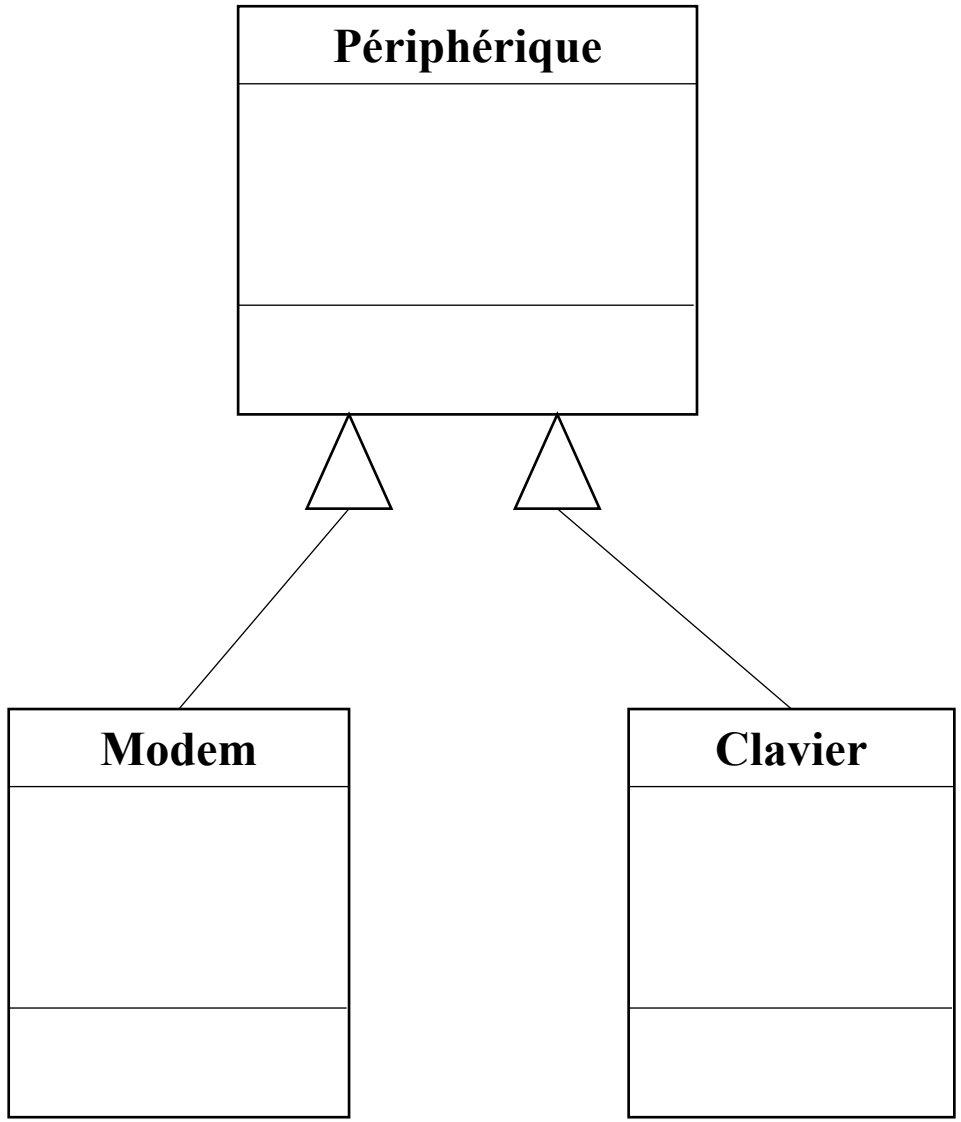
4- Un compte bancaire peut appartenir à une personne physique ou morale.

Correction:

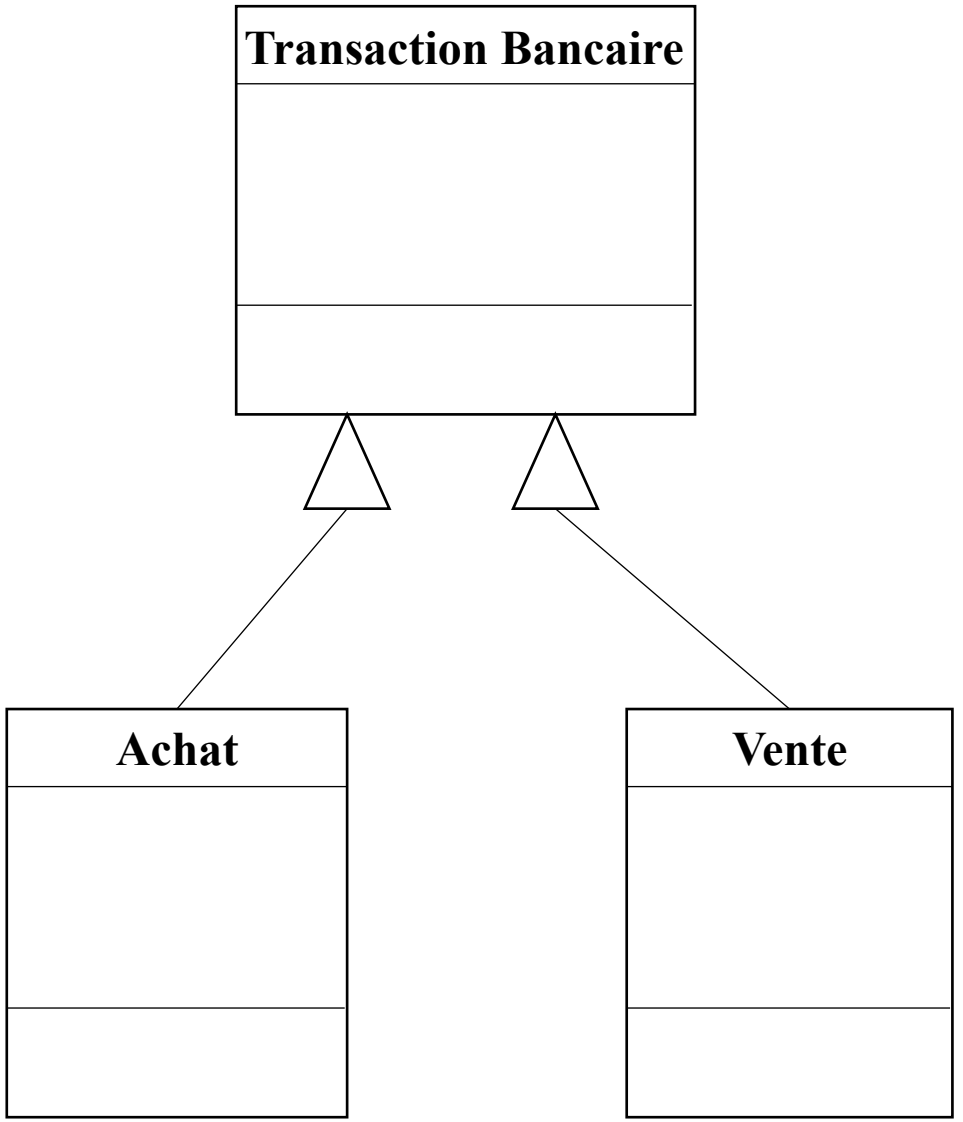
1-



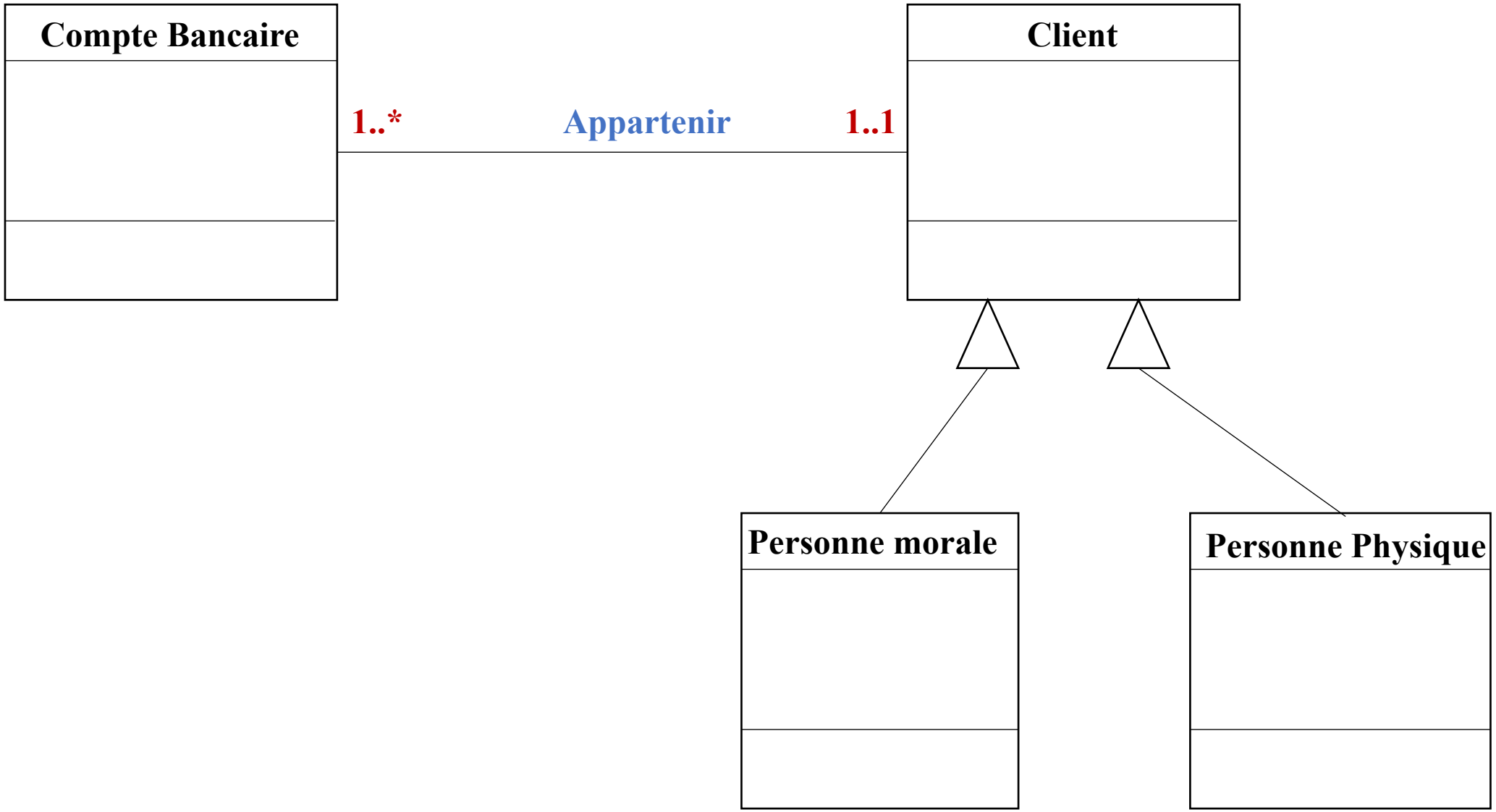
2-


Périphérique
Modem
Clavier

3-


Transaction Bancaire
Achat
Vente

4-


Compte Bancaire
1..*
Appartenir
1..1
Client
Personne morale
Personne Physique

Chapitre 4:Le diagramme D'objets